1 Lesson 03-11-26

1.1 I2C

We know move from *SPI* to **I2C** (*Inter Integrated Circuit*, also known as IIC).

I2C allos for multiple masters, multiple slaves, where:

- Only masters can initiate a transfer;
- Multiple masters require arbitration.

It also requires only 2 wires:

- **SCLK** for the clock;
- **SDATA** for data: a single data line, for half duplex communication.

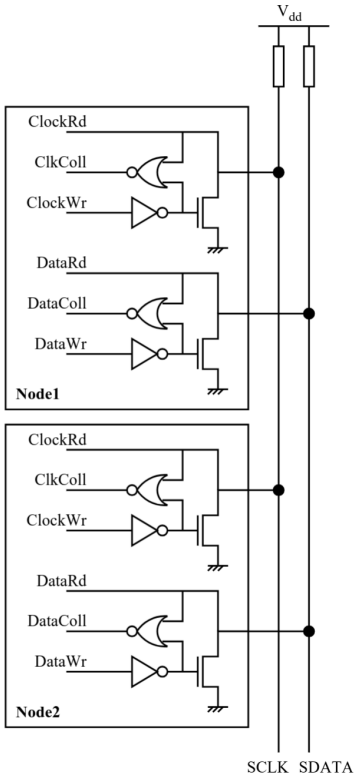
As always, the 2 wire solution (SPI was 3, but still separate clock) introduces distance limits due to skew. Also, no select signals for slaves means the protocol requires addressing: this introduces a certain level of *overhead* in the system.

All devices can read and write the SDATA and SCLK lines:

- The SCLK line mainly controlled by masters, slaves can only stretch the clock duration if they are slow (cooperative clock).
- The SDATA line controlled by a single transmitter (master or slave) at any given time

The protocol should avoid collisions: an *Open Drain* (pull-up) architecture is used for both SCLK and SDATA. If lines are:

- Logic 0: the line is driven to GND by the low side output transistor of a node;
- Logic 1: the low side transistor is OFF, the line is left floating. An external pull-up resistor brings the line to the high voltage.



This means that when I2C is idle, both lines are normally high.

The architecture allows for detecting collisions on SCLK and SDATA.
A collision happens when a node generates 0, another generates 1.

- The output nMOS transistor of the node generating 0 is ON.
- The output nMOS transistor of the node generating 1 is OFF.

There is no short circuit, no damages to the system, and the line just goes to 0 (the nMOS outputting 0 wins).

All nodes constantly monitor the SCLK and SDATA lines:

- The device generating the 1 detects the collision;
- The device generating the 0 does not notice.

Two kinds of collisions, actually features:

- On the SCLK line we have *clock stretching*;
- On the SDATA line we have *arbitration*.

1.1.1 I2C addressing

Each slave in an I2C network is assigned a unique **address** on 7 bits. This means we have a maximum of 128 slaves, but typically much less. Nonetheless, there exists a protocol extension to 10-bit addresses.

Some addresses are reserved, and masters do not need an address.

The address is often hardwired in a slave by connecting pins to V_{dd} or ground. Some bits are not configurable (usually the MSBs) to save on the number of pins on the slave. The master sends the address of the target slave at the beginning of a transmission, to address it.

1.1.2 I2C protocol

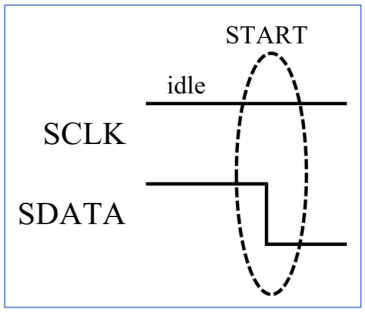
SCLK and SDATA lines are normally high when there is no transmission: this just means that all low side nMOS output transistors are OFF.

The main rule is that the SDATA line is allowed to change only when SCLK is low. Two exceptions mark frames:

- **START** configuration: SDATA makes a high to low transition while SCLK is high;
- **STOP** configuration: SDATA make a low to high transition while SCLK is high.

Now:

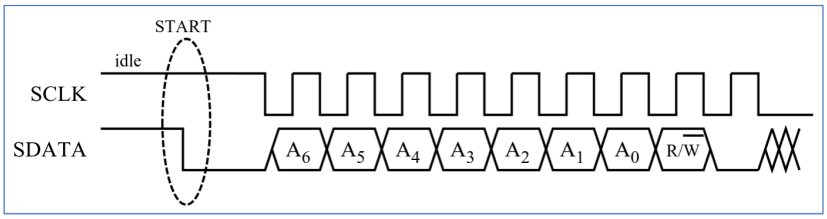
1. All communications begin with a START configuration;



2. An addressing phase follows: the master activates the clock and generates on SDATA the target slave address on 7 bits, MSB first, one bit per clock cycle.

One more bit is needed for the R/W flag (R when high). Then we get one low bit from the slave before releasing the SDATA line (the ACK).

SDATA usually changes only when SCLK is low, shortly after the falling edge of SCLK. Slaves samples SDATA immediately after the rising edge of SCLK.

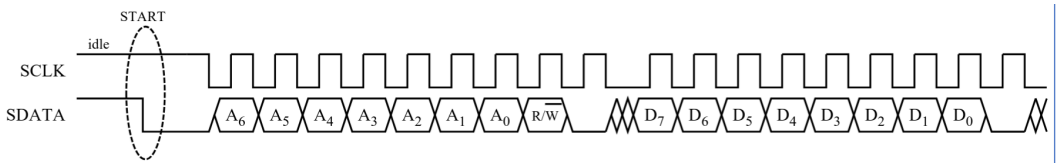


3. A data exchange phase in a single direction is next. We said that the master generates 9 clock cycles: 8 bits for data, 1 bit for ACK.

   Now the situation changes depending on the type of operation the master initiates:

   - *Write*:
     - The master regains control of SDATA and generates 8 data bits, MSB first;
     - The slave generates ACK on the last bit (SDATA=0).
   - *Read*:
     - The slave keeps control of SDATA and generates 8 data bits, MSB first;
     - The master generates ACK on the last bit (SDATA=0).

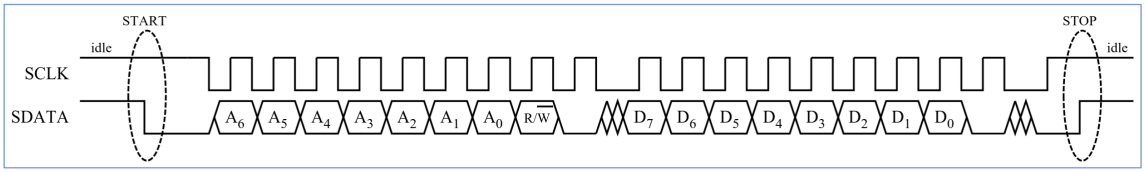
   This way, only a byte is transferred. Multiple bytes can be transferred one after the other, with no limit;



4. A STOP configuration concludes the communication.

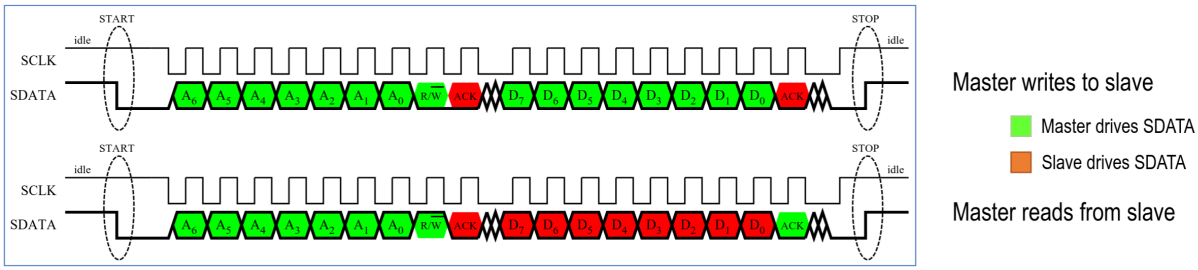
   After the last data byte, the master (or the slave, on writes) keeps SDATA low (the ACK).

   Then, the master itself keeps SDATA low while releasing the SCLK: SCLK goes high. When the master then releases SDATA as well, SDATA makes a low to high transition, which signals the STOP configuration.



The bus is now idle again. Hooray.

Lastly, let's note the difference between a read and a write operation performed by the master:



Some other rules:

- The first 8 bits are driven by the transmitter (TX), always a master for the addressing phase, a master or a slave in the data phase;
- The ACK bit is driven by the receiver (RX), always a slave for the addressing phase, a master or a slave in the data phase;
- Before and after ACK the SDATA line may show oscillations while control passes from TX to RX and viceversa. Oscillations only happen while SCLK is low, and SDATA must become stable before the rising edge of SCLK;
- An ACK after the addressing phase means the slave is ready for transmission. If the ACK is missing (i.e. SDATA=1), the transmission is immediately aborted;
- An ACK after the data phase, writing to slave means the slave received the data. It doesn't mean it was received correctly, as there is no error detection on bits. If the ACK is missing, the data definitely was not received correctly;
- An ACK after the data phase, reading from slave means the master is ready to receive an additional byte. If the ACK is missing, no more bytes are requested by the master.

The opposite of an ACK (leaving the SDATA rail up) is called a NACK.

A **RESET** configuration is needed as sometimes a master needs to write a command to a slave, and then immediately read an answer.

This normally requires two transmissions, the first in writing, the second in reading. If the master doesn't want other masters to gain control of the bus before the second (reading) transmission, at the end of the first (writing) transmission, it can:

- Not generate a STOP configuration;
- Generate a START configuration, again.

This means that the master sets SDATA high, releases SCLK, sets SDATA low, which signals the RESTART configuration. The bus never becomes idle, thus no other master can gain control.

1.1.3 Clock stretching

Let's have a look at the first case in which we use collision detection. We focus on collision detection on the SCLOCK line, that is **clock stretching**.

Normally the clock on SCLK is generated by a master, for all bits: a slave should be sufficiently fast to operate at the clock frequency generated by the master. In this case, the slave only receives the clock and never writes it.

A slow slave may extend the time at which the clock is low, simply by activating its own nMOS transistor on the SCLK line, while it is already forced low by the master. When the master releases SCLK, the line remains low: the master detects the collision and waits for the SCLK line to be released by the slave, as well.

The communication then proceeds as usual.

If clock stretching is used, individual bits may have different bit time. Note that not all masters and slaves support clock stretching.

1.1.4 Arbitration

This is the second mechanism in which collision detection is used. We focus on collision detection on the SDATA line, that is **arbitration**.

Multiple master may wish to start a transmission at the same time. If the bus is busy, a master shall not start a transmission. If two masters Master1 and Master2 (or more) concurrently generate a START configuration, they won't notice it, as they are both driving the same line low: each master believes he is the only owner of the bus!

We assume in the following time that they generate the same clock, and during the addressing phase, they (probably) generate two different addresses. If at a certain bit Master1 generates a 0 and Master2 generates a 1, that address line goes low: Master1 does not notice any problem, Master2 detects a collision on SDATA. At this point Master2 has lost the contention and stops the transmission, so that Master1 can complete the transmission.